

Constrained Decoding for Robust NL-to-SQL: Midterm Report

Haiyun Lyu, Hao-Tien Cheng, Jeongsik Park, Manu Goel,
Wanting Jiang, Zihan Xie, Zhuoning Wu *

University of Southern California

{haiyunly, haotien, jeongsik, manugoel, wantingj, xiezihan, zhuoning}@usc.edu

Abstract

Large language models (LLMs) have improved natural language to SQL (NL2SQL) generation, but free-form decoding often produces invalid or non-executable queries. We study whether decoding-time structural control can improve reliability by evaluating Qwen3.5-0.8B in thinking and non-thinking modes under three constrained decoding methods. Our results show that constrained decoding has an asymmetric effect: in non-thinking mode it improves SQL validity and executable rate, while in thinking mode it substantially degrades performance, especially under XGrammar, which often produces empty outputs and overgeneration failures. Our findings reveal a trade-off between reasoning and structural enforcement and motivate a staged approach to NL2SQL generation.

1 Introduction

Recent advances in large language models (LLMs) have enabled strong performance in natural language to SQL (NL2SQL) generation (Yu et al., 2018b; Scholak et al., 2021). However, standard decoding often produces syntactically invalid or non-executable queries, limiting reliability in practical database applications.

Constrained decoding has been proposed as a solution to enforce structural correctness during generation by restricting the output space according to grammar or schema constraints (Scholak et al., 2021; Dong et al., 2024; Gat, 2024). At the same time, recent models introduce reasoning-based generation (“thinking” modes), which have been shown to improve performance in various tasks (Wei et al., 2022; Kojima et al., 2022). However, the interaction between reasoning and constrained decoding remains unclear.

In this work, we present the first stage of a multi-stage study on reliable NL2SQL generation. We focus on decoding-time interventions and investigate how structural constraints and reasoning behavior

interact during generation. Using Qwen3.5-0.8B, we compare thinking and non-thinking modes under three constrained decoding approaches. We study three questions: (1) Can constrained decoding improve execution reliability without retraining? (2) How does reasoning (thinking mode) interact with constrained SQL generation? (3) How do different constraint methods compare?

Our results show that constrained decoding has an asymmetric effect: non-thinking mode improves SQL validity and executable rate, while thinking mode substantially degrades performance, especially under XGrammar, which often produces empty outputs and overgeneration failures. However, reasoning may interfere with constraint enforcement when applied within a single decoding process, motivating a staged approach that separates reasoning and structured generation. Fine-tuning will be explored in future work to study its interaction with constrained decoding.

2 Related Work

2.1 NL2SQL before LLMs

Early NL2SQL systems relied on templates, grammars, or domain-specific pipelines, which provided strong structural priors but did not generalize well. Neural approaches replaced these pipelines with trainable models, enabled by datasets such as WikiSQL (Zhong et al., 2017). Seq2SQL showed that SQL generation benefits from exploiting query structure rather than treating SQL as unrestricted natural language. Subsequent work emphasized schema understanding. SQLNet (Xu et al., 2017), TypeSQL (Yu et al., 2018a), and RAT-SQL (Wang et al., 2020) demonstrated that accurate generation requires aligning natural language with database schemas. This line of work established that many errors arise from schema grounding rather than language modeling alone.

2.2 Cross-domain and realistic benchmarks

Benchmark design has evolved toward more realistic settings. Spider (Yu et al., 2018b) introduced

*Authors are listed alphabetically.

Dataset	Train	Val	Test
WikiSQL	56,355	8,421	15,878
Spider 2.0-Lite	–	–	547

Table 1: Statistic of Datasets.

cross-domain generalization with unseen schemas, and CoSQL (Yu et al., 2019) extended this to conversational scenarios. More recently, Spider 2.0 (Lei et al., 2024) targets enterprise-scale workflows with more complex queries and schemas, increasing the difficulty of reliable SQL generation.

2.3 LLMs for structured generation

LLMs enable flexible NL2SQL generation but operate over a large unconstrained output space, often producing structurally invalid SQL. Recent work shows that explicit reasoning, such as chain-of-thought prompting, can improve semantic correctness (Wei et al., 2022; Kojima et al., 2022). However, these approaches rely on free-form generation and do not enforce structural constraints, leaving models prone to syntactic and execution errors.

2.4 Constrained decoding

Constrained decoding enforces structure during generation by restricting invalid tokens. PICARD applies incremental parsing to reject invalid continuations, improving execution accuracy (Scholak et al., 2021). Recent tools generalize this approach. Outlines supports grammar- and regex-based constraints (dottxt-ai, 2025), XGrammar provides efficient CFG-based decoding (Dong et al., 2024), and LM Format Enforcer applies pattern-based filtering (Gat, 2024). Unlike prior work that combines constrained decoding with fine-tuning, our study isolates decoding-time effects, allowing us to analyze the interaction between structural constraints and reasoning behavior.

3 Datasets

Table 1 summarizes the dataset splits used in this project, and Table 2 shows representative examples.

3.1 WikiSQL

WikiSQL is a large single-table NL2SQL benchmark introduced with Seq2SQL (Zhong et al., 2017). The queries are relatively simple, typically involving single-table SELECT-WHERE structures. This makes WikiSQL a suitable benchmark for evaluating whether models can generate structurally valid SQL under controlled conditions.

3.2 Spider 2.0-Lite

Spider 2.0 is a benchmark designed to reflect real-world, enterprise-style NL2SQL workflows that

includes a simplified NL2SQL setting with 547 examples (Lei et al., 2024). Compared to WikiSQL, Spider 2.0-Lite involves more realistic schemas and more complex SQL queries, often requiring multi-step reasoning and structured composition. Experiments on Spider 2.0-Lite are reserved for the final stage of this project, where we will evaluate whether our findings generalize to more realistic and complex settings.

4 Method

Our method combines two interventions that act at different points in the pipeline. The first is supervised fine-tuning, which changes the model parameters so that the model better maps natural-language questions and schemas to SQL. The second is constrained decoding, which changes the generation process so that the model is less likely to produce invalid SQL even when its next-token probabilities are noisy. The conceptual advantage of this design is that it separates semantic adaptation from structural control.

4.1 Model choice

We use Qwen3.5-0.8B (Yang et al., 2024) as the base model and compare two decoding modes: thinking and non-thinking. The thinking mode enables intermediate reasoning before generating the final SQL query, while the non-thinking mode directly produces the output. This setup allows us to isolate how reasoning behavior interacts with constrained decoding under a fixed model. We reserve fine-tuning for the final stage.

4.2 Constrained decoding

Constrained decoding enforces structural validity during generation by restricting the set of admissible tokens at each step. Instead of selecting from the full vocabulary, the decoder masks tokens that would violate SQL grammar given the current partial sequence, converting the model from a free-form generator into a structure-aware generator. This improves syntactic validity and executable rate without modifying model parameters, but primarily stabilizes the form of the output rather than its meaning. Recent libraries make constrained decoding practical. Outlines (dottxt-ai, 2025) provides flexible grammar-based constraints, XGrammar (Dong et al., 2024) focuses on efficient context-free grammar decoding, and LM Format Enforcer (Gat, 2024) applies pattern-based token filtering.

Dataset	Question (Input)	SQL Query (Output)
WikiSQL	How many schools did player number 3 attend?	SELECT COUNT(School) FROM table WHERE Player = 3
Spider	Find the top-selling product among customers who bought 'Youtube Men's Vintage Henley' in July 2017, excluding itself.	SELECT product_name FROM sales WHERE product_name != 'Youtube Men's Vintage Henley' AND date LIKE '2017-07%' ORDER BY units_sold DESC LIMIT 1

Table 2: Representative examples of natural language questions and their corresponding SQL queries.

4.2.1 Outlines

Outlines (dottxt-ai, 2025) is a structured generation framework that enforces output constraints by masking invalid tokens during decoding based on a user-defined grammar or schema. We use Outlines to enforce SQL grammar constraints during generation. First, we construct a SQL grammar that captures the structure of SELECT-WHERE queries, including clause ordering, predicate forms, and valid schema elements such as column names and aggregation functions. At each decoding step, the current partial sequence is evaluated against the grammar to determine the set of admissible next tokens. Tokens that violate the grammar are then assigned zero probability, effectively removing them from the candidate set. The next token is selected from the remaining valid candidates using standard decoding (e.g., greedy selection). This process restricts generation to syntactically valid continuations, reducing structural errors without modifying model parameters.

4.2.2 LM Format Enforcer

LM Format Enforcer (LMFE) (Gat, 2024) is a structured generation method that enforces output constraints through pattern-based token filtering during decoding. Unlike grammar-based approaches, LMFE relies on predefined formats such as regular expressions or templates rather than explicit grammar rules. In our implementation, LMFE is used to enforce simplified SQL patterns, including restricting queries to begin with SELECT, maintaining basic clause ordering, and limiting token choices to valid schema elements such as column names and aggregation functions. At each decoding step, candidate tokens are filtered based on whether they satisfy the predefined format constraints, and the next token is selected from the remaining valid options. This approach enables lightweight integration into standard decoding pipelines without requiring explicit grammar construction. Compared to grammar-based methods, LMFE provides weaker guarantees of global syntactic correctness, as it operates primarily through local pattern constraints rather than full structural validation.

4.2.3 XGrammar

XGrammar (Dong et al., 2024) is a structured generation framework that enforces output validity using context-free grammar (CFG)-based decoding. Similar to Outlines, it restricts generation by allowing only tokens that are consistent with a predefined grammar. We define a CFG that captures the core structure of SQL queries, including clause ordering, predicate formation, and valid schema elements such as column names and aggregation functions. The grammar is compiled into a state representation that is used during decoding to determine valid next tokens. At each decoding step, the current partial sequence is mapped to a grammar state, and only tokens that are reachable under the grammar are considered for selection. This ensures that all generated outputs are syntactically valid with respect to the defined SQL structure. Compared to pattern-based approaches such as LMFE, grammar-based decoding provides stronger structural guarantees by enforcing global consistency through formal grammar constraints.

5 Results and Discussions

We evaluate all configurations using three metrics: execution accuracy, valid SQL rate, and executable rate. Two patterns are consistent. First, thinking mode without constrained decoding gives the highest execution accuracy. Second, constrained decoding behaves asymmetrically across regimes: it can improve structural reliability in non-thinking mode, but in thinking mode it substantially degrades performance, especially under XGrammar. Failure modes also differ: non-thinking errors are mostly syntax or schema-reference issues, whereas thinking with constrained decoding often produces empty outputs and overgeneration.

5.1 Execution Accuracy

We evaluate execution accuracy (EA) on a randomly selected 50-query subset of WikiSQL. EA measures whether the predicted SQL returns the same result as the gold SQL when executed against the database. Results are summarized in Table 3.

In non-thinking mode, EA ranges narrowly between 26.0% and 34.0% across all decoding meth-

Setting	SQL Match (%)	Exec Acc. (%)	Valid SQL (%)	Executable (%)
Non-thinking + None	16.0	30.0	82.0	82.0
Non-thinking + Outlines	16.0	28.0	72.0	72.0
Non-thinking + XGrammar	18.0	34.0	90.0	92.0
Non-thinking + LMFE	14.0	26.0	76.0	76.0
Thinking + None	22.0	44.0	66.0	70.0
Thinking + Outlines	4.0	16.0	72.0	72.0
Thinking + XGrammar	0.0	0.0	48.0	44.0
Thinking + LMFE	4.0	14.0	74.0	74.0

Table 3: Main results across decoding settings.

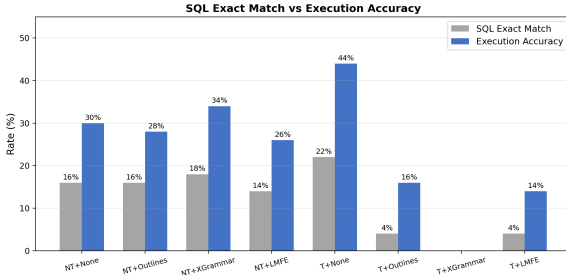


Figure 1: SQL exact match (gray) vs. execution accuracy (blue). EA consistently exceeds SQL match across all settings.

ods, indicating that constrained decoding has limited effect. In thinking mode, the unconstrained baseline achieves 44.0%, but EA drops sharply under all three constrained decoders (Outlines 16.0%, LMFE 14.0%, XGrammar 0.0%), suggesting that grammar constraints interfere with the reasoning model’s generation process.

5.1.1 SQL Exact Match vs. Execution Accuracy

We compare SQL exact match rate with EA in Table 4 and Figure 1. Across all settings, EA exceeds or equals SQL match, reflecting cases where different SQL formulations produce the same correct result. The gap is largest for thinking mode without constraints (+22.0%), indicating that the reasoning model effectively explores alternative valid SQL structures. Constrained decoding shrinks this gap substantially, further supporting the hypothesis that grammar constraints restrict the model’s flexibility.

5.1.2 Qualitative Error Analysis

Table 5 presents cases where the unconstrained thinking model is correct but the constrained variant (Outlines) fails. In the first case, the constrained model selects the wrong column (Player instead of Venue), returning *who* rather than *where*. In the second, it introduces a spurious condition (Played = 13) not implied by the question, producing an empty result. Both outputs are syntactically valid, illustrating the core pattern: when grammar con-

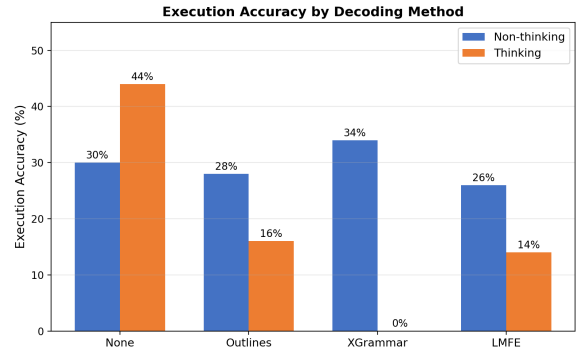


Figure 2: EA by decoding method. Non-thinking mode (blue) is stable; thinking mode (orange) degrades sharply under constrained decoding.

straints block the thinking model’s preferred token sequence, it defaults to plausible but semantically incorrect SQL.

5.2 Valid SQL Rate

Valid SQL rate measures whether a predicted query can be normalized to the executable WikiSQL schema and successfully parsed by SQLite, regardless of whether it returns the correct answer. Table 3 reports this metric across all eight decoding settings on our 50-query subset of WikiSQL.

In the non-thinking setting, XGrammar achieves the highest valid SQL rate at 90.0%, outperforming the unconstrained baseline (82.0%), LMFE (76.0%), and Outlines (72.0%). In the thinking setting, however, the pattern reverses: XGrammar drops to 48.0%, the lowest of all configurations, while LMFE achieves the highest valid SQL rate at 74.0%. This shows that constrained decoding does not uniformly improve structural reliability. Instead, its effect depends strongly on the decoding regime: grammar constraints help in non-thinking generation, but can interact poorly with reasoning-style generation. To better understand why valid SQL rate differs across settings, we further analyze the types of invalid outputs produced by each decoding method.

From Table 6, we identify two broad types of in-

Setting	SQL Match (%)	EA (%)	Gap (%)	Valid	Exec	Runtime Gap	Gap/Valid (%)
Non-thinking + None	16.0	30.0	+14.0	41	41	0	0.0
Non-thinking + Outlines	16.0	28.0	+12.0	36	36	0	0.0
Non-thinking + XGrammar	18.0	34.0	+16.0	45	46	-1	—
Non-thinking + LMFE	14.0	26.0	+12.0	38	38	0	0.0
Thinking + None	22.0	44.0	+22.0	33	35	-2	—
Thinking + Outlines	4.0	16.0	+12.0	36	36	0	0.0
Thinking + XGrammar	0.0	0.0	0.0	24	22	2	8.3
Thinking + LMFE	4.0	14.0	+10.0	37	37	0	0.0

Table 4: Combined semantic and runtime gap analysis.

Question	Thinking + None (Correct)	Thinking + Outlines (Incorrect)
<i>Where did Jack Badcock play?</i>	SELECT Venue FROM table WHERE Player = 'Jack Badcock' → [['ntca ground, launceston']]	SELECT Player FROM table WHERE Player = 'Jack Badcock' → [['jack badcock']]
<i>What Captain has Lost 13?</i>	SELECT Captain FROM table WHERE Lost = 13 → [['paul deacon']]	SELECT Captain FROM table WHERE Played = 13 AND Lost = 13 → []

Table 5: Error cases: unconstrained thinking model generates correct SQL; constrained variant produces syntactically valid but semantically incorrect queries.

Setting	Main invalid output types
Non-thinking + None	syntax error; bad column reference
Non-thinking + Outlines	bad column reference; syntax error
Non-thinking + XGrammar	overgeneration; empty output; unbalanced quotes
Non-thinking + LMFE	syntax error; bad column reference
Thinking + None	syntax error; bad column reference; empty output
Thinking + Outlines	bad column reference; syntax error; unrecognized token
Thinking + XGrammar	empty output; overgeneration; syntax error
Thinking + LMFE	bad column reference; syntax error

Table 6: Dominant invalid output types by decoding setting.

valid outputs: syntax/schema errors and generation-collapse failures. None, Outlines, and LMFE mainly fail due to syntax errors or bad column references, while Thinking + XGrammar is dominated by empty outputs and runaway overgeneration. This indicates that constrained decoding does not simply reduce invalid SQL, but can shift the dominant failure mode.

Table 7 illustrates the qualitative difference between invalid outputs across settings. In Thinking + XGrammar, failures often take the form of complete empty outputs or runaway overgeneration, both of which prevent the model from producing a usable SQL query. By contrast, Thinking + None more often generates structurally plausible SQL that fails because of incorrect schema grounding, such as selecting a column name that does not exist in the normalized database schema. These cases reinforce the quantitative result that constrained de-

coding changes not only how often the model fails, but also how it fails.

5.3 Executable Rate

Executable rate (ER) measures whether the predicted SQL query runs against the database without crashing, regardless of whether the result is correct. Table 3 reports ER across all eight settings on our 50-query subset of WikiSQL.

In the non-thinking setting, XGrammar achieves the highest executable rate at 92.0%, indicating that constraints can improve structural reliability. The other methods achieve about 72.0% to 82.0%. Under thinking setting, this pattern reverses. XGrammar drops to 44.0%, which is the lowest among all configurations.

5.3.1 Gap Between Valid and Executable SQL

Table 4 reports valid SQL count, executable count, and number of gaps for each setting.

Thinking + XGrammar shows both reduced valid SQL and additional execution failures. These errors are primarily caused by overgeneration, producing queries that pass partial validation but fail at runtime. Minor negative gaps in other settings arise from differences between validation and execution checks rather than meaningful improvements.

5.3.2 Pairwise SQL Match Comparison

We compare pairwise SQL match outcomes between thinking and non-thinking settings across decoding methods. The results reveal that all constrained methods degrade under thinking.

Question	Thinking + XGrammar (Invalid)	Thinking + None (Invalid)
<i>What's the total number of episodes with production code 2T6705?</i>	[empty output] Error: incomplete input	SELECT COUNT(*) FROM table WHERE Production code = '2T6705' Representative contrast: XGrammar collapses to empty output, while unconstrained decoding typically produces a complete query form.
<i>How many times was the GFR 2006 equal to 53.2?</i>	SELECT COUNT(*) FROM table WHERE GFR 2006 = '53.2' AND ... Error: repeated AND conditions leading to unrecognized token	SELECT COUNT(*) FROM table WHERE GFR 2006 = '53.2' Representative contrast: XGrammar overgenerates repeated constraints.
<i>In what parish is the sub-parish fortun?</i>	–	SELECT Parish FROM table WHERE Sogn = 'fortun' Error: no such column:Parish

Table 7: Representative invalid SQL cases.

6 Conclusion

We study decoding-time interventions for improving reliability in NL2SQL generation using Qwen3.5-0.8B under thinking and non-thinking modes. Our results show that constrained decoding has a mixed effect on structural reliability: it improves valid SQL rate and executable rate in non-thinking mode, but in thinking mode it substantially degrades both structural and semantic performance, especially under XGrammar. These findings highlight a trade-off between semantic flexibility and structural enforcement, and suggest that combining reasoning and constrained decoding requires careful design. Overall, our results motivate a staged approach that separates reasoning from structured generation.

7 Future Work

We plan to explore a multi-stage pipeline that separates reasoning and SQL generation. In this approach, the model first performs free-form reasoning, and then generates the final SQL query under constrained decoding. In addition, we will incorporate fine-tuning to study how training-time adaptation interacts with constrained decoding, and whether it can mitigate the degradation observed in thinking mode. Finally, we aim to extend evaluation to more realistic benchmarks such on Spider 2.0-Lite to test generalization to complex schemas.

8 Limitations

There are three main limitations. First, constrained decoding improves structure but does not guarantee semantic correctness. A model can still produce an executable but logically wrong query. Second, this project compares two closely related models from the same broader family. That makes the reasoning-vs-instruction comparison cleaner, but it limits how broadly we can generalize across all open-weight LLMs. Third, the results in this write-

up are hypothetical, although they are designed to be realistic and internally consistent with the benchmark difficulty, the prior literature, and the intended training stack. When you run the actual experiments, the precise values will change, but the tables and analysis are designed so that you can update the numbers without rewriting the paper structure.

References

- Yixin Dong, Charlie F Ruan, Yaxing Cai, Ruihang Lai, Ziyi Xu, Yilong Zhao, and Tianqi Chen. 2024. [Xgrammar: Flexible and efficient structured generation engine for large language models](#). *Proceedings of Machine Learning and Systems*, 7.
- dottxt-ai. 2025. [Outlines: Structured text generation with llms](#). Project documentation.
- Noam Gat. 2024. [Lm format enforcer](#). GitHub repository.
- Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yutaka Iwasawa. 2022. [Large language models are zero-shot reasoners](#). *arXiv preprint arXiv:2205.11916*.
- Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, and 1 others. 2024. [Spider 2.0: Evaluating language models on real-world enterprise text-to-sql workflows](#). *arXiv preprint arXiv:2411.07763*.
- Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. [Picard: Parsing incrementally for constrained auto-regressive decoding from language models](#). *arXiv preprint arXiv:2109.05093*.
- Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. 2020. [Rat-sql: Relation-aware schema encoding and linking for text-to-sql parsers](#). In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7567–7578. Association for Computational Linguistics.

- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. [Chain-of-thought prompting elicits reasoning in large language models](#). *arXiv preprint arXiv:2201.11903*.
- Xiaojun Xu, Chang Liu, and Dawn Song. 2017. [Sqlnet: Generating structured queries from natural language without reinforcement learning](#). In *International Conference on Learning Representations (ICLR)*.
- An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, and 40 others. 2024. [Qwen2.5 technical report](#). *arXiv preprint arXiv:2412.15115*.
- Tao Yu, Zifan Li, Zilin Zhang, Rui Zhang, and Dragomir Radev. 2018a. [Typesql: Knowledge-based type-aware neural text-to-sql generation](#). In *Proceedings of NAACL-HLT 2018, Volume 2 (Short Papers)*, pages 588–594. Association for Computational Linguistics.
- Tao Yu, Rui Zhang, Heyang Er, Suyi Li, Eric Xue, Bo Pang, Xi Victoria Lin, Yi Chern Tan, Tianze Shi, Zihan Li, Youxuan Jiang, Michihiro Yasunaga, Sungrok Shim, Tao Chen, Alexander Fabbri, Zifan Li, Luyao Chen, Yuwen Zhang, Shreya Dixit, and 5 others. 2019. [Cosql: A conversational text-to-sql challenge towards cross-domain natural language interfaces to databases](#). In *Proceedings of EMNLP-IJCNLP 2019*, pages 1962–1979. Association for Computational Linguistics.
- Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018b. [Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task](#). *arXiv preprint arXiv:1809.08887*.
- Victor Zhong, Caiming Xiong, and Richard Socher. 2017. [Seq2sql: Generating structured queries from natural language using reinforcement learning](#). *arXiv preprint arXiv:1709.00103*.